

Why is software OS-Specific?

Or: How to write cross-platform software

Team name: NetworkStack

Emre Acarsoy, Luca Croci, Kazybek Khairulla, Luca Pacitti, Harry Taylor

University of Exeter

26/02/2026

Problem Statement

- ▶ An **operating system** is fundamental for computers to function.
- ▶ **Problem:** Many programs can only be run on certain operating systems.
- ▶ **Key Example:** Adobe Photoshop, used by over 80,000 companies worldwide [1], is incompatible with Linux.
- ▶ Why does this problem exist? How can we resolve this?

Overview of the OS

- ▶ An operating system is the software that **controls the execution of applications.**
- ▶ **Interface** between user programs and computer hardware.
- ▶ Responsible for managing I/O, processes, memory...
- ▶ The OS provides access to: [2]
 - ▶ kernel functionality through **system calls**
 - ▶ displaying graphics through **graphics APIs**

System calls

	Windows	Unix
Spawning a process	CreateProcess()	fork() + exec()
Terminating a process	ExitProcess()	exit()
Creating a file	CreateFile()	open()
Closing a file	CloseHandle()	close()
Process management	GetCurrentProcessID()	getpid()
Set file permissions	SetFileSecurity()	chmod()

Table: Comparison of Windows vs Unix system calls [3] [4] [5]

Graphics APIs

Overview

- ▶ Interface between software and the GPU.
- ▶ Graphics APIs are used for displaying visuals [6].

Problem

- ▶ Many popular graphics APIs are not available on all major operating systems.
- ▶ For cross-compatibility, developers must rewrite rendering layers for other APIs [7].

Graphics API	Windows	macOS	Linux
DirectX	✓	—	—
Metal	—	✓	—
Vulkan	✓	—	✓
OpenGL	✓	✓	✓

Assessing Solutions – User experience

Criteria for assessing solutions:

- ▶ **Stability** - Is the program stable? Do crashes occur?
- ▶ **Performance** - Does the system lose any performance compared to bare metal? How much performance is lost?
- ▶ **Features** - Are any features missing compared to running to the bare metal? What features?
- ▶ **Availability of Updates** - What support can the system expect to have? How do users access updates?

Assessing Solutions – Developer Experience (DX)

- ▶ **Development time** - How much additional development time is necessary?
- ▶ **Additional expertise** - Development teams must be knowledgeable in the cross-compatibility solution they use.
- ▶ **Maintenance** - How much additional maintenance time and cost is incurred?

Plan for the next section

We will go over the following 3 solutions:

1. Translation layers (namely, Wine and Proton)
2. Virtualisation
3. Electron

For each of these, we will:

- ▶ give an overview
- ▶ go over the advantages and disadvantages
- ▶ evaluate how well they meet our criteria (DX & UX)

Solutions - Translation layers - Wine and Proton

Wine ("Wine Is Not an Emulator")

- ▶ Wine is an open source compatibility layer for running Windows software on unix-like systems (MacOS, Linux) [8]
- ▶ Works based on reverse engineering the Windows kernel [8]
- ▶ Runs on the fly in user space, using wineserver daemon [8]

Proton

- ▶ Proton is a fork of wine bundled with other software, such as VKD3D, a translation layer for DirectX → Vulkan [9]
- ▶ Proton runs out of the box (using Steam Play) [10]

Wine and Proton - Advantages and Disadvantages

Advantages

- ▶ Minimal input from the developer to port their system using wine or proton
- ▶ Lower performance overhead as wine/proton acts as a translation layer, and not a full virtual machine
- ▶ Open source

Disadvantages

- ▶ Apps often don't work out of the box, requiring configuration (improved with proton)
- ▶ Not all software runs perfectly, if at all

Wine and Proton - Evaluation

User experience

- ▶ **Stability:** Stability is high, with occasional crashes for misconfigured/incompatible applications
- ▶ **Performance:** < 15% effect on performance [11]
- ▶ **Features:** Lack of driver support and limited anti-cheat [12]
- ▶ **Updates:** Major updates every year [13]

Developer experience

- ▶ **Development Time:** Small increase with Wine specific testing
- ▶ **Additional Expertise:** Wine debugging and winetricks configs
- ▶ **Maintenance:** Testing for each major wine version

Solutions - Virtualisation

Virtualisation

- ▶ Divides physical computer resources into a series of virtual environments. [14]
- ▶ Software directly shares a host machine's hardware.
- ▶ This is NOT the same as emulation! [14]

Containerisation

- ▶ Virtualisation at OS-level. [15]
- ▶ Software, along with its prerequisite OS libraries and dependencies, is packaged into a container. [15]
- ▶ **Container:** A single executable that can run consistently on ANY system infrastructure. [15]

Containerisation - Advantages and Disadvantages

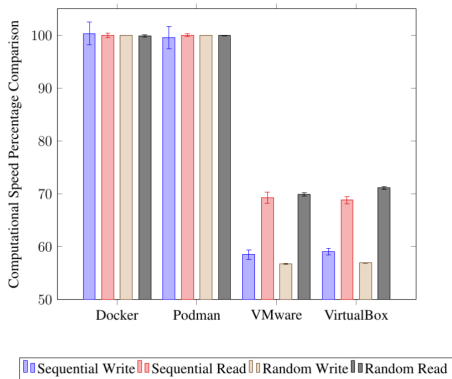
Advantages

- ▶ Improves scalability and utilisation of resources [16]
- ▶ Container overhead $<$ VM overhead [16]
- ▶ Host's kernel shared by all containers [17]
- ▶ Each container isolated during its runtime [17]

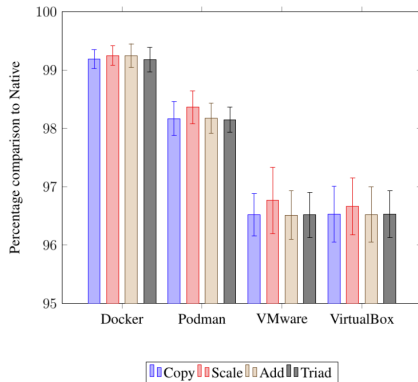
Disadvantages

- ▶ Kernel vulnerability $\rightarrow$ security risk to all containers [18]
- ▶ Complex to manage large volumes of containers
- ▶ Suboptimal performance for resource-intensive workloads [16]

Containerisation vs VMs - Performance comparison



(a) Memory throughput



(b) Performance benchmark

Figure: Performance comparisons between containerisation (Docker, Podman) and VMs (VMware, VirtualBox) [19] [20] [21]

Containerisation - Evaluation

User experience

- ▶ **Stability** - Very stable
- ▶ **Performance** - Excellent due to fewer overheads
- ▶ **Features** - Highly portable
- ▶ **Updates** - Container must be replaced in case of an update [22]

Developer experience

- ▶ **Development Time** - Efficient once set up
- ▶ **Additional Expertise** - Helpful to understand container tools
- ▶ **Maintenance** - Low, but management ideal at scale

Solutions - Electron

Electron

- ▶ Open source framework for building cross-platform desktop applications with a JavaScript runtime [23]
- ▶ 1 language for Windows, Linux and MacOS [23]
- ▶ Translates web interfaces into desktop software rather than translating system calls

Examples

- ▶ VS Code, Discord, Slack

Electron - Advantages and Disadvantages

Advantages

- ▶ Cross-platform support for Windows, Linux, and MacOS
- ▶ Shared web and desktop code → code reusability
- ▶ Open source

Disadvantages

- ▶ High memory usage → lower performance compared to native applications
- ▶ Large file sizes → Chromium and Node.js are bundled with every application

Electron - Evaluation

User experience

- ▶ **Stability:** Can degrade in unoptimised applications
- ▶ **Performance:** Complex layouts and heavy animations increase CPU and GPU usage
- ▶ **Features:** Poor support for low-latency tasks (e.g: trading)
- ▶ **Updates:** Frequent updates through built-in updaters, often involving large downloads

Developer experience

- ▶ **Development Time:** Efficient
- ▶ **Additional Expertise:** Helpful to understand Electron and web development languages
- ▶ **Maintenance:** Moderate

References I

- [1] J. Cass, “Adobe statistics – users, revenue, growth more (feb 2025),” Accessed: Feb. 23, 2026. [Online]. Available: <https://justcreative.com/adobe-statistics/>.
- [2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 1.00. Arpaci-Dusseau Books, Aug. 2018.
- [3] M. Kerrisk, “Syscalls - linux system calls,” Accessed: Feb. 18, 2026. [Online]. Available: <https://man7.org/linux/man-pages/man2/syscalls.2.html>.
- [4] K. Michael, *The Linux Programming Interface*. No Starch Press, 2010.
- [5] M. Santos, “Understanding the windows system call mechanism,” Accessed: Feb. 20, 2026. [Online]. Available: <https://medium.com/@int2Eh/understanding-the-windows-system-call-mechanism-7d27db219478>.
- [6] Wikipedia Contributors, “Graphics library,” Accessed: Feb. 23, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Graphics_library.

References II

- [7] D. Szabó and Z. Illés, “High performance gpu graphics api abstraction layer in c# for real-time graphics,” *Procedia Computer Science*, vol. 235, pp. 1258–1267, 2024.
- [8] Wikipedia, “Wine (software),” Accessed: Feb. 22, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Wine_\(software\)](https://en.wikipedia.org/wiki/Wine_(software)).
- [9] Wikipedia, “Proton (software),” Accessed: Feb. 22, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Proton_\(software\)](https://en.wikipedia.org/wiki/Proton_(software)).
- [10] Valve Corporation, “Introducing a new version of steam play,” Accessed: Feb. 22, 2026. [Online]. Available: <https://steamcommunity.com/games/221410/announcements/detail/1696055855739350561>.
- [11] M. Kopel and M. Bożek, “Is proton good enough? - a performance comparison between gaming on windows and linux,” in *Computational Collective Intelligence*, N. T. Nguyen et al., Eds., Cham: Springer Nature Switzerland, 2023, pp. 634–646.

References III

- [12] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,” in *6th Symposium on Operating Systems Design & Implementation (OSDI 04)*, San Francisco, CA: USENIX Association, Dec. 2004. [Online]. Available: <https://www.usenix.org/conference/osdi-04/recovering-device-drivers>.
- [13] The Wine Project, “Wine,” version 11.3, Accessed: Feb. 22, 2026. [Online]. Available: <https://gitlab.winehq.org/wine/wine>.
- [14] Wikipedia, “Virtualization,” Accessed: Feb. 23, 2026. [Online]. Available: <https://en.wikipedia.org/wiki/Virtualization>.
- [15] O. Bentaleb, A. S. Belloum, A. Sebaa, and A. El-Maouhab, “Containerization technologies: Taxonomies, applications and challenges,” *The Journal of Supercomputing*, vol. 78, no. 1, pp. 1144–1181, 2022.

References IV

- [16] L. Baresi, G. Quattrocchi, and N. Rasi, “A qualitative and quantitative analysis of container engines,” *Journal of Systems and Software*, vol. 210, p. 111965, 2024.
- [17] K. Wang et al., “Characterizing and optimizing kernel resource isolation for containers,” *Future Generation Computer Systems*, vol. 141, pp. 218–229, 2023.
- [18] M. Sroor, T. Das, R. Mohanani, and T. Mikkonen, “A systematic mapping study on risks and vulnerabilities in software containers,” *arXiv preprint arXiv:2512.11940*, 2025.
- [19] L. Hyryläinen, “Performance comparison of different virtualization methods on commercial off the shelf hardware,” *Master’s thesis, University of Oulu*, 2024.
- [20] A. Kopytov, “Sysbench: Scriptable database and system performance benchmark,” Accessed: Feb. 22, 2026. [Online]. Available: <https://github.com/akopytov/sysbench>.

References V

- [21] AMD, *Stream benchmark*, Accessed: Feb. 22, 2026. [Online]. Available: <https://www.amd.com/en/developer/zen-software-studio/applications/spack/stream-benchmark.html>.
- [22] Docker, “Immutability — docker docs,” Accessed: Feb. 23, 2026. [Online]. Available: <https://docs.docker.com/dhi/core-concepts/immutability/>.
- [23] OpenJS Foundation and Electron Contributors, “Why electron,” Accessed: Feb. 23, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/why-electron>.

Individual contribution allocation

Team member	Contribution
Emre Acarsoy	25%
Luca Croci	15%
Kazybek Khairulla	10%
Luca Pacitti	25%
Harry Taylor	25%